PHÂN TÍCH NÂNG CẤP THUẬT TOÁN TÌM ĐƯỜNG

DFS → IDA*

Tìm đường có chi phí tối ưu bằng ngưỡng f, chấp nhận tìm lại qua nhiều vòng.

DFS nền tảng

Đi sâu bằng **stack LIFO**; gặp đích thì dừng. Tập explored tránh xử lý lại trạng thái đã xét.

Hạn chế Không xét ưu tiên chi phí nên không đảm bảo đường rẻ nhất. Bản tích hợp bỏ cạnh cost = ∞ để không đi qua đường cấm.

Trên Romania ngập 9 node, cost 1.335,80; gấp 2,03 lần tối ưu.

IDA* theo ngưỡng f

Lặp DFS với **f = g + h**. Cắt nhánh có f vượt ngưỡng; vòng sau dùng f nhỏ nhất vừa bị cắt.

Chỉ giữ path hiện tại và on_path để tránh cycle; **không dùng visited toàn cục**. h Euclid được kiểm tra tính nhất quán trên cạnh.

Điều kiện Chi phí không âm, h admissible; node được đếm lại qua mỗi vòng.

Quá trình nâng cấp

DFS
Một lượt đi sâu theo thứ tự láng giềng.

+ Heuristic và ngưỡng
f = g + h; tăng ngưỡng thay vì giữ OPEN lớn.

+ Chi phí ngập
c = d·(1 + α·q/Cv); q: độ ngập. q ≥ Cv: c = ∞.

Đường tối ưu Arad → Sibiu → Rimnicu → Pitesti → Bucharest
Cost 657,85; 11 vòng; 63 expanded; path tối đa 5 node, gồm đích.

Kết quả đo lại bằng wrapper chung

Romania: seed 42, Cv = 60, α = 2, 7 lần đo. Lưới 8×8 ngập thập phân: 1 lần đo (*). A* ref là tham chiếu.

Đầu vào	Solver	Cost	Expanded	Time (ms)	Peak (KB)
Romania ngập	DFS	1.335,80	9	0,182	4,77
Romania ngập	**IDA***	657,85	63	1,197	8,05
Romania ngập	A* ref	657,85	11	0,228	4,91
Lưới 8×8 (*)	DFS	246,90	15	0,286	7,97
Lưới 8×8 (*)	**IDA***	204,93	2.417.004	63.273,077	16,49
Lưới 8×8 (*)	A* ref	204,93	63	0,985	14,48

Best case và giới hạn RAM

Hành lang thẳng có 100 node chính và 100 nhánh cụt: h chính xác trên trục chính. IDA* chỉ cần **1 vòng, 100 expanded**.

Ít vòng chưa đảm bảo ít RAM: IDA* 71,23 KB so với A* 48,99 KB. Lưới khô 10×10 (*) giảm RAM khoảng 21%.

Worst case có nhiều ngưỡng

Lưới có cycle, h Euclid thấp hơn chi phí đi đường và nhiều f gần nhau. IDA* phải tìm lại: **2.168 vòng, 2.417.004 expanded**.

Lần đo có tracemalloc mất 63,3 giây. Bộ nhớ tìm kiếm theo độ sâu khi b cố định; graph đầu vào vẫn được lưu riêng.

IDA* cải thiện chất lượng lời giải so với DFS; lợi ích RAM và chi phí CPU phụ thuộc đầu vào.

Time và peak Python đo cùng lượt; median. KB = bytes/1024, không phải tổng RAM. (*) Một mẫu. Nguồn: dữ liệu chạy lại; Korf 1985 và Vempaty et al. 1991.

PHÂN TÍCH NÂNG CẤP THUẬT TOÁN TÌM ĐƯỜNG

UCS → Bi-UCS

Hai phía thu hẹp vùng tìm kiếm; chi phí dựng graph đảo và quản lý state vẫn phải tính.

UCS nền tảng

Dùng min-heap theo **g**, từ start đến goal. Lưu best cost và parent; bỏ entry cũ. Với cost không âm, goal được lấy ra có chi phí tối ưu.

Romania khô Cost 418, 13 expanded. Hòa chi phí dùng thứ tự FIFO.

Giới hạn Có thể mở rộng một vùng lớn trước khi chạm đích.

Bi-UCS hai hướng

Chạy từ start trên graph gốc và từ goal trên **graph đảo**. Mỗi phía có heap, g và parent; cập nhật μ là đường tốt nhất đã nối được.

Dừng an toàn min(gF) + min(gB) ≥ μ sau khi bỏ entry cũ; không dừng ở lần chạm đầu.

Expanded Tổng forward và backward. Cạnh cost = ∞ bị bỏ qua.

Quá trình nâng cấp

Một UCS từ start
Một làn sóng theo chi phí g tăng dần.

+ UCS từ goal
Đảo hướng cạnh, giữ nguyên trọng số.

+ Cận dừng μ
Ghép parent hai phía và trả đường tối ưu.

Cùng đường tối ưu cost 418 Romania khô: UCS 13 expanded; Bi-UCS 10 = 5 + 5.
Bản đo bao gồm dựng graph đảo trong solve; giảm node không đảm bảo giảm thời gian.

Kết quả đo lại trên các cấu trúc khác nhau

7 lần/test thông thường; đồ thị random 20.000 node dùng 1 lần (*). Cùng input, cost và chính sách đo trong mỗi cặp.

Đầu vào	Solver	Cost	Expanded	Time (ms)	Peak (KB)
Romania khô	UCS	418,00	13	0,183	4,27
Romania khô	**Bi-UCS**	418,00	10	0,314	10,52
Lưới mở 31×31	UCS	10,00	221	2,460	42,81
Lưới mở 31×31	**Bi-UCS**	10,00	82	4,685	275,55
Random 20k (*)	UCS	27,00	8.262	391,945	5.104,11
Random 20k (*)	**Bi-UCS**	27,00	220	180,917	8.374,68
Hub có hướng	UCS	51,00	52	0,495	11,09
Hub có hướng	**Bi-UCS**	51,00	3.051	70,407	1.389,88

Best case có phân nhánh

Random 20k: **8.262 → 220 expanded**, giảm 97,3%; time còn 0,46 lần UCS (*).

Lưới mở cũng giảm node, nhưng tốn thêm time và RAM cho graph đảo và hai phía.

Worst case lệch hai phía

Hub có 3.000 cạnh đi vào: tìm ngược phình lớn, **3.051 so với 52 expanded**.

Chuỗi 2.000 node hầu như không lợi: 1.999 so với 2.000 expanded; thêm bookkeeping.

Bi-UCS có lợi khi hai phía cùng phân nhánh; cần cân cả node, thời gian và bộ nhớ.

Time/peak đo cùng lượt, có tracemalloc; graph đầu vào dựng sẵn. (*) Một mẫu. Nguồn: dữ liệu chạy lại; Goldberg et al., tài liệu Princeton về bidirectional Dijkstra.

PHÂN TÍCH NÂNG CẤP THUẬT TOÁN TÌM ĐƯỜNG

A* → D* Lite

Tái sử dụng g, rhs và hàng đợi để sửa đường khi chi phí ngập hoặc start thay đổi.

A* chạy lại từ đầu

Min-heap theo **f = g + h**; bỏ entry cũ. Khi bản đồ hoặc start thay đổi, chạy A* mới trên snapshot mới.

Baseline thật Dùng solve_astar trong bài nộp Phương Anh; A* ref của nhóm DFS chỉ dùng đối chiếu.

Cùng điều kiện Graph, cost ngập và heuristic Euclid được kiểm tra ở mọi phase.

D* Lite giữ state để repair

Tìm ngược từ goal; giữ **g, rhs, U**. Heap dùng khóa kép theo min(g, rhs); xử lý đỉnh chưa nhất quán.

Cost đổi: cập nhật nguồn cạnh. Start di chuyển: tăng **k_m**. Sửa đến khi start nhất quán.

Cùng một planner xuyên suốt episode; expanded đếm riêng từng phase.

Quá trình nâng cấp và phép đo

A* theo snapshot
Mỗi lần chạy dựng lại tìm kiếm.

+ Trạng thái g và rhs
Giữ thông tin từ lần tìm trước.

+ Update và repair
Đo cả cập nhật, di chuyển và tìm lại.

Romania động Arad → Sibiu; chặn Rimnicu-Pitesti rồi giảm ngập Fagaras-Bucharest.
Mỗi path/cost được đối chiếu Dijkstra; cùng graph và chuỗi cập nhật ở hai thuật toán.

Initial và từng repair trên Romania

7 lần đo/phase, median; cùng tracemalloc. Time initial gồm khởi tạo; time repair gồm update + move + plan. Peak giữ state từ trước.

Đầu vào	Solver	Cost	Expanded	Time (ms)	Peak (KB)
Initial	A*	657,85	11	0,237	5,02
Initial	D* Lite	657,85	8	0,582	13,12
Repair 1	A*	639,18	5	0,093	5,02
Repair 1	D* Lite	639,18	5	0,223	15,59
Repair 2	A*	584,32	5	0,085	5,73
Repair 2	D* Lite	584,32	2	0,104	16,99

Best case: sửa cục bộ

Lưới 8×8, 8 lần cập nhật gần start: riêng repair giảm **422 → 85 expanded**.

Tổng time repair: A* 7,39 ms; D* Lite 5,20 ms. Peak episode cao hơn do giữ state; giảm node chưa đủ kết luận về RAM.

Worst case: ít tái sử dụng

Ngập sát goal trên lưới 7×7 làm lan truyền sửa đổi: cả episode A* **144**, D* Lite **147 expanded**.

Lưới tĩnh 7×7: A* 0,78 ms, D* Lite 1,96 ms. D* Lite trả overhead khi không có lượt repair để tái sử dụng.

D* Lite có lợi khi thay đổi cục bộ; chi phí dương và heuristic hợp lệ là điều kiện của adapter.

Peak: Python allocations, gồm planner và output phase trước; KB = KiB, không phải RSS. Nguồn: code Phương Anh; kết quả chạy lại; Koenig & Likhachev 2002 và errata.